

Received 23 March 2025, accepted 12 April 2025, date of publication 21 April 2025, date of current version 12 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3562612

RESEARCH ARTICLE

iYOLOV7-TPE-SS: Leveraging Improved YOLO Model With Multilevel Hyperparameter Optimization for Road Damage Detection on Edge Devices

HANIAH MAHMUDAH^{1,2}, (Member, IEEE), AULIA SITI AISJAH², (Member, IEEE), SYAMSUL ARIFIN², (Member, IEEE), AND CATUR ARIF PRASTYANTYO³

¹Department of Electrical Engineering, Politeknik Elektronika Negeri Surabaya (Electronics Engineering Polytechnic Institute of Surabaya), Surabaya 60111, Indonesia

²Engineering Physics Department, Institut Teknologi Sepuluh Nopember, Surabaya 60111, Indonesia

³Civil Engineering Department, Institut Teknologi Sepuluh Nopember, Surabaya 60111, Indonesia

Corresponding author: Aulia Siti Aisjah (auliasa@its.ac.id)

This work was supported in part by the Politeknik Elektronika Negeri Surabaya (PENS), and in part by the Institut Teknologi Sepuluh Nopember (ITS).

ABSTRACT Digital twin cities integrated road damage inspection to support smart road infrastructure. The road's condition was assessed using road damage survey metrics, and the issue was resolved manually. Real-time, automatically, and precisely employing artificial intelligence (AI) is needed to improve road safety and reduce maintenance costs. This research on road defect detection utilizes improved YOLOV7 (iYOLOV7) while multilevel hyperparameter optimization using combination of the Tree-Structured Parzen Estimator (TPE) and Search Space (SS). This model we call TPE-SS. The iYOLOV7 provides real-time processing on edge devices that have limited storage, low-speed processors, and memory. The research investigates how leveraging multilevel hyperparameter optimization using the TPE-SS model improves system performance where road damage is detected. The results of iYOLOV7 model demonstrate the smallest total loss of 0.1, indicating a significant performance improvement where a precision value of 0.986, an average recall (AR) of 0.970, a mean average precision mAP@0.5 of 0.988, a mean average precision mAP@0.5-0.9 of 0.806, and an F1-score of 0.978. The measurement results and analysis findings reveal that our proposed method is accurate enough. The embedded system employs NVIDIA Jetson Nano for inference, which takes only 0.135 seconds and has a scalability performance of 7.470 FPS to recognize things associated with road damage. Although the employed NVIDIA AGX Orin edge device may produce a higher FPS of 67.034 and a faster inference time of 0.014 seconds. This means that the iYOLOV7 model is extremely feasible and practical for usage on edge devices to identify road damage.

INDEX TERMS iYOLOV7 model, multilevel hyperparameter optimization, tree-structured parzen estimator, search space, scalability performance, road damage.

I. INTRODUCTION

Developing a digital replica of a city as a digital twin of a city in a variety of ways, including monitoring building damage, road damage, smart road and congestion to improve

The associate editor coordinating the review of this manuscript and approving it for publication was Cesar Vargas-Rosales¹.

functional urban infrastructure, hence improving safety, comfort, and economic development to encourage smart cities [1] [2]. Utilizing an integrated road defect inspection system that incorporates smart roads equipped with real-time and automated road detection technologies. Digital twin cities with road damage inspection as a support for smart roads can be observed in the form of system road damage,

which comprises sensors such as accelerometer, laser, LiDAR, cameras, GPS, data communication devices, and edge devices coupled to hardware to develop a road damage detection system. Raw data from sensors is transformed into valuable insights through visualization data, such as road damage and position in virtual entities. Road defect detection generates data, which is sent to the server via an internet-based wireless connection. IoT devices with wireless connectivity can send data to a centralized server by utilizing connectivity various wireless technologies and security data encryption protocols. The server generates analysis, modeling, monitoring, optimization, and big data road damage as a digital twin service that is used for a real-time and automatic continuous monitoring system, resulting in more efficient road monitoring.

Real-time and automated road detection technologies are required to increase road safety, lower maintenance costs, and enhance overall road infrastructure performance. This device would help in detecting and fixing road faults in a timely way, preventing future deterioration and accidents. The road condition evaluation can be employed to identify problems including cracks, potholes, and surface irregularities [3], [4], [5].

The road condition evaluation is used to decide the type of road damage management program. The type of road maintenance treatment is determined by examining the pavement's functional condition. Three methodologies are used to measure pavement condition: the International Roughness Index (IRI), the Pavement Condition Index (PCI), and the Surface Distress Index (SDI) [6], [7], [8].

In contrast, SDI is a commonly used parameter. SDI method for determining the functional state of the pavement. The SDI examines three types of road damage: potholes, cracks, and ruts. SDI has the advantage of making it simple and easy for surveyors to visually measure the functional state of the road in a typical manner. On the other side, one disadvantage of SDI is that its simplicity might lead to inaccuracies in estimating the functional condition of the road, resulting in an incorrect selection of the type of road treatment [9], [10]. Furthermore, it necessitates the hiring of several field surveyors as well as the allocation of funds for road damage maintenance.

The road's condition is evaluated using road damage survey metrics. The problem of collecting road damage survey data is tackled manually. This requires a substantial amount of surveyor effort and funds for road damage maintenance. That is compounded since the number of professionals that are able to assess such road damage is limited, and secondly, methods commonly employed for gathering information from the field are laborious, cost-intensive, require a non-trivial level of skill, and are highly subjective and prone to errors [10].

Automating the road defect detection process with a device can provide maintenance teams with real-time data, allowing them to prioritize repairs and allocate resources more

effectively. Devices that identify road faults automatically and in real time typically have two major procedures: road data collection and road defect identification. When implementing a real-time road defect detection system, it should consider the precision and speed of detection as well as the inference time.

Road data acquisition is the process of gathering data about the road surface condition using various sensors and cameras. Accelerometers [11], [12], [13], [14], [15], [16], 3D reconstruction techniques with vision-based stereo utilizing cameras, lasers, LEDs, and LiDAR (light detection and ranging) [17], [18], [19], [20] and cameras are some examples of sensors used in research. The data obtained may include criteria like road roughness, texture, cracks, potholes, and other anomalies. This data collection process is frequently continuous and occurs in real time as the device moves through the road network.

When road data is acquired, these devices evaluate the data using algorithms based on image processing techniques, machine learning, and deep learning to identify instances of road problems, a process known as road defect identification. Several studies have used camera sensors, image processing [21], [22], [23], [24], [25], [26], [27], machine learning (ML) [28], [29], [30], [31] and deep learning [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44], [45], [46], [47], [48] approaches to detect road damage. These algorithms are developed to detect patterns associated with various types of damage, such as cracks, potholes, and surface deformations.

The road damage detection research makes use of laser sensors to rebuild 3D road damage as well as stereo vision technologies. Several sensors are required for the stereo vision system, including two cameras, an LED, a laser, and LiDAR. The laser sensor has a small beamwidth; thus, it cannot detect road damage in the form of a large hole, necessitating the use of an array of LEDs, lasers, LiDARs, and other sensors [18], [49]. Reconstructing the pothole takes time, whether using an LED array, laser, or LiDAR. Furthermore, 3D road defect reconstruction and stereo vision require well-lit lighting and slow vehicle speeds [17], [18], [50].

Further research used image-processing techniques. Potholes are a form of road irregularity that can be recognized with traditional image processing techniques. To achieve high accuracy, this technique takes numerous steps, including manual image feature extraction and image processing parameter adjustments. As a result, the image frame rate varies depending on traffic circumstances [21]. Some research that investigated road conditions, pavement conditions, potholes, cracks, smart phone sensors, and accelerometers used machine learning techniques such as support vector machines, neural networks, multilayer perceptron, and decision trees [16]. Furthermore, research is being conducted to detect pavement damage using ML algorithms such as support vector machines, artificial neural networks, and

random forests [51]. To improve accuracy, ML models require significant computer resources during data processing, which also includes manual feature analysis.

Several studies used deep convolutional neural network (CNN) techniques to automate the simultaneous feature extraction and categorization of objects [41], [52]. This is challenging to incorporate into real-time road defect detection systems. The deployment of CNN models for road damage detection on devices requires a balance between high accuracy and high efficiency [53]. The use of CNN models for road damage detection on devices requires achieving a balance between high accuracy and high efficiency. The use of CNN models for road damage detection on devices necessitates striking a balance between high accuracy and high efficiency. The parameter system performance deployment model CNN on devices performs well in terms of precision, recall, F1-score, and as well as efficiency of time inference.

The CNN models perform exceptionally well in terms of high accuracy, precision, recall, and low loss. This requires CNN models with large architecture and competitive computing costs that are inefficient. To do this while maintaining high accuracy, a simplified CNN model configuration setting is required to achieve good CNN model system performance. High efficiency, which is the CNN model system performance, enables it to detect road faults on edge devices with a short inference time. Because the CNN model on edge devices have limited storage capacity, inference time must be considered. Taking this into account, the CNN model offers good system performance and a short inference time for simpler model configurations, but high accuracy is difficult to achieve. To obtain optimal system performance, the CNN model parameters must be properly chosen. When selecting a detection model, there is no definitive answer as to which model has the best performance [31], [53].

Improving system performance in computer vision, object recognition, and classification using CNN models can be achieved through various methods. These include modifying or adding components to the CNN model structure and optimizing hyperparameter selection during CNN model training. The strategy of enhancing the CNN structure aims to improve feature extraction efficiency and model accuracy by modifying or incorporating new components into the CNN architecture. Several studies have focused on enhancing CNN models by modifying or adding components to the existing CNN structure [54], [55], [56], [57], [58], [59]. DenseNet was integrated into the YOLOv4 backbone to optimize feature transfer and reuse. This involved several modifications, such as adjusting the Path Aggregation Network (PANet) [54], introducing New Residual Blocks, applying Spatial Pyramid Pooling (SPP), modifying the PANet, and utilizing Hard-Swish activation functions [59]. Similarly, researchers have integrated DenseNet blocks into the YOLOv5 backbone to enhance the preservation and reuse of critical feature information. This was achieved by incorporating a Convolutional

Block Attention Module (CBAM) [56]. Another study introduced a novel framework called Multiple Graph Learning Neural Networks (MGLNN) for semi-supervised learning on graph-based data [55]. Additionally, modifications were made to the YOLO model using the CSPDarknet53 backbone structure, which included Residual Blocks, DenseNet, Spatial Pyramid Pooling (SPP), and the Path Aggregation Network (PANet) [57].

When training a CNN model, optimizing hyperparameter selection can significantly enhance system performance. Key CNN model parameters that can be optimized include Kernel size, activation function selection, learning rate and scheduler optimization, normalization and regularization techniques, and optimizer selection. Numerous researchers have explored hyperparameter optimization (HPO) techniques. For instance, systematic reviews have been conducted on HPO methods for CNN models [60], while others have focused on hyperparameter optimization techniques based on Bayesian optimization, [61], and hyperparameter optimization with Bayesian approaches [62]. The researchers [63] employed multiscale and multilevel evolutionary optimization (MSMLEO) and Gaussian process-based Bayesian optimization (GPEI) to optimize dozens of hyperparameters. Common approaches to hyperparameter optimization include grid search, random search, and Bayesian optimization. Among these, the Tree-structured Parzen Estimator (TPE), a Bayesian optimization method, has gained popularity and is widely used in modern parameter tuning frameworks [64]. Additionally, the importance of hyperparameter optimization and accelerated multi-objective optimization has grown, particularly with the use of the Multi-Objective Tree-Structured Parzen Estimator (MOTPE) [65], [66]. Furthermore, various HPO strategies have been integrated to address challenges in CNN models, such as modified adaptive TPE [67] and the combination of grid search with TPE [68]. Some research results on hyperparameter optimization methods show that these techniques can greatly increase the performance of CNN model systems.

Some research on road damage identification utilizing images with CNN models employs some pre-trained architectural CNN one-stage detector models, namely InceptionV2, MobileNet [33], [34], [48] and some You Only Look Once (YOLO) models [42], [69]. For example, Z. S. Hernanda [34] employed the pre-trained MobileNet V2 model with tuning hyperparameter of batch size variable changes, resulting in a mAP of 0.08 and an AR of 0.241. Meanwhile, Maeda [33] utilizes the MobileNet V2 model without hyperparameter tuning, which is used on smartphones and has a precision of 0.71 and an inference time of 1.5 seconds. Masoud Faramarzi [42] employs the DarkFlow-YOLO model, which includes batchsize and optimizer variation parameters and yields nearly zero results in training and validation accuracy and average precision. The DarkFlow-YOLO model was unable to detect road damage.

However, CNN models in [33] and [42] were detect road damage and tested on cellphones rather than one of the edge devices, the NVIDIA Jetson. The CNN models that have already been deployed on the NVIDIA Jetson require accurate improvements; however, they have not taken into consideration model inference time on the NVIDIA Jetson. And most importantly, earlier research did not take into account the implementation of CNN models in real time on devices with limited storage; therefore, the complexity of CNN models influences the performance of accuracy systems (mAP and AR) as well as efficiency systems (inference time), as shown Fig. 1. To close this gap and address the constraints of real-time implementation of CNN models on devices with limited storage, this study seeks to create an improved YOLOV7 (iYOLOV7) model by multilevel hyperparameter optimization using a combination of the Tree-Structured Parzen Estimator (TPE) and space search, which is the iYOLOV7-TPE-SS model. The iYOLOV7-TPE-SS model employs hyperparameters, learning rates, batch sizes, and optimizer parameters. The pre-trained iYOLOV7-TPE-SS model achieves optimal system performance while deployed on edge devices using NVIDIA Jetson.

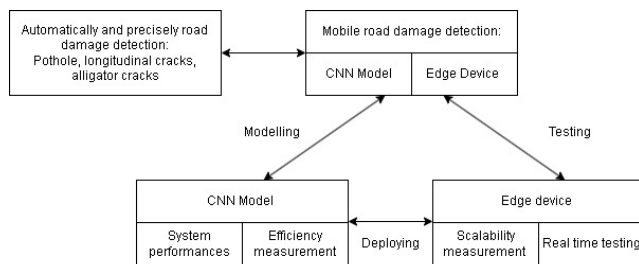


FIGURE 1. Mobile road damage detection.

The use of batch size variations and optimizers aims to find the most superior YOLO model with two system performance parameters: high accuracy in mAP, AR, and F1-scores and efficiency in system performance, namely the inference time of the model detecting road damage objects on the complexity of the CNN model. The optimizer parameters were successfully applied to the RetinaNet152 model for the road damage problem [70]. However, in this study, this model will be utilized to improve the pre-trained YOLOV7 model's [39] ability to detect potholes.

Employing CNN models for road damage detection on edge devices saves energy due to limited store capacity by delivering good system performance, such as high accuracy or low error. This results in large costs for analyzing each CNN model parameter, making manual adjustment of CNN model hyperparameters impracticable. Several studies on tuning hyperparameters of CNN model [34], [35] and hyperparameter optimization [38] using Bayesian methods have yielded positive results for system performances. However, most of these strategies were developed without taking into account CNN models for identifying road damage when deployed on edge devices with limited storage.

The study provides a road detection system that uses hyperparameter tuning of the iYOLOV7-TPE-SS model to increase performance. The iYOLOV7-TPE-SS variant is deployed on edge devices and can identify road damage in real time. This study examines system performance and efficiency. The iYOLOV7-TPE-SS model is capable of detecting road damage on edge device in real time. The primary contributions of this research can be stated in the following points.

- We develop an improved YOLOV7 (iYOLOV7) model to detect potholes and enhance other road faults by leveraging multilevel hyperparameter optimization using a combination of the TPE and SS, which is the TPE-SS model. The proposed iYOLOV7 with TPE-SS (iYOLOV7-TPE-SS) model is deployed on edge devices, allowing them to automatically detect road damage.
- The iYOLOV7-TPE-SS model for road damage was developed to detect potholes and enhance other road faults by incorporating alligator cracks and longitudinal cracks. To improve the system's performance, the iYOLOV7-TPE-SS model by obtaining the loss function, changing the batch size parameter, and optimizing the most excellent system performance as expressed by the parameters of mean average Precision (mAP), recall, and F1 score.
- The iYOLOV7-TPE-SS model is being deployed on embedded edge devices. Efficiency and scalability measurements across several embedded edge devices are achieved. This study builds a road damage system utilizing the iYOLOV7-TPE-SS model on embedded edge devices using NVIDIA Jetson Nano and NVIDIA Jetson AGX Orin with system performance assessed in inference time and frame per second (FPS) as efficiency performances.
- The iYOLOV7-TPE-SS model is deployed on embedded edge devices to detect road challenges in real time utilizing mobile. We retrieve real-time measurements to detect road damage. Mobile road damage measurement is performed with a camera sensor connected to an edge device deploying the iYOLOV7-TPE-SS model.

The structure of writing begins with an introduction section I and a related work section II. Next, Section III, which is the proposed system for road damage detection, contains strategies to improve system performance and increase efficiency, consisting of dataset retrieval, dataset pre-processing, and the iYOLOV7-TPE-SS model using multilevel hyperparameter optimization using the Tree-Structured Parzen Estimator and Search Space. Furthermore, Section IV covers performance measurement and comprehensive analysis. It presents results of system performance measurement, efficiency measurement, scalability measurement on edge devices, and comparison of CNN models on edge device which are thoroughly assessed using the measurement result data. The last section includes the conclusions and future

research on the development of road damage identification technologies.

II. RELATED WORKS

In the literature, road data acquisition refers to the process of gathering road surface condition data using cameras and deep learning to identify instances of road damage, also known as the difficulty of road damage detection. Many deep learning-based techniques, including CNN models, have been utilized in the literature to address the issues of recognizing road defects. Some researchers utilize manual hyperparameter tuning and optimization strategies to ensure that the CNN model performs optimally.

Maeda et al. [33] utilized the SSD-InceptionV2 and SSD-MobileNet models. The dataset collection includes bounding boxes for each of eight types of road damage. The data collection includes the bounding boxes for each of the eight forms of road damage. Each image is derived from an image set that was made by taking images of a large number of roads with a vehicle-mounted smartphone. The SSD-InceptionV2 and SSD-MobileNet model frameworks were loaded on a smartphone and used with a GPU on a personal computer (PC), with recall and precision of 71% and 77%, respectively, with an inference time of 1.5 seconds on a smartphone device. Faramarzi [42] processed the same dataset [33] using different architectures, batch sizes, and optimizers. The CNN model designs employed were YOLO, tiny-YOLOv2, Darknet Neural Networks, YOLOv3, tiny-YOLOv3, and YOLOv4 algorithms, with a mean average precision (mAP) of 45%, or 0.45, and a loss of less than 1.5. As a result, training and validation accuracy and average precision are nearly zero.

Zhang et al. [37] examined the performance of road defect classification using the CrackNet II model with different kernel sizes. The CrackNet II model has 10 layers: four convolution layers, five 1×1 convolution layers, and one output layer. The planned CrackNet II includes two significant changes to the original CrackNet. First, CrackNet II no longer uses the feature generator, which delivers the collaborative features of ten developers using a predetermined technique that cannot be developed. As a result, all of the layers in CrackNet II contain learnable parameters that may be adjusted throughout the training process. Second, CrackNet II has a more complex architecture, with more hidden levels but fewer settings. All layers employ the same image width and height to ensure that each pixel's mistake is explicitly taught. For faster convergence, hyperparameters, smaller batch sizes, and a higher learning rate are used first. Later, refining was carried out using a larger batch size and a lower learning rate. CrackNet II obtained 90.20% overall accuracy, 89.06% recall, and 89.62% F-measure on 200 test photos. When compared to the original CrackNet, CrackNet II can detect more fine cracks or hairlines while reducing more local sounds and retaining significantly faster processing performance.

Pham et al. [44] test Detectron2's implementation of Faster R-CNN with various base models and parameters on the crack dataset. The results suggest that the X101-FPN basic model for faster R-CNN with Detectron2's default configurations is efficient and general enough to be applicable to multiple nations in this challenge. This method yields F1 scores of 51.0% and 51.4% for test1 and test2 sets of the challenge, respectively. Although the visualizations produce reasonable prediction results, the F1 scores are poor. As a result, their studies compare the prediction findings to the current annotations, identify some inconsistencies, and recommend techniques to improve the tagging process for this datasets.

Liu et al. [38] developed a novel YOLOv3 model with a ResNet50vd-deformable convolution (DCN) backbone and a Bayesian search-based hyperparameter optimization (HPO) strategy. It employs nondestructive ground-penetrating radar (GPR) testing to accurately identify and locate concealed cracks in asphalt pavements. First, it investigates concealed cracks in the asphalt surface to generate a crack stress dataset consisting of 366 images and 533 cracks. To estimate the mean average precision (mAP) and the best appropriate hyperparameters using the semi-supervised label distillation (SSLD) approach and HPO-based Bayesian search. Using the SSLD method, the experimental results reveal that the YOLOv3-ResNet50vd-DCN model converges faster and has a lower loss value (approximately 0.05) during the training phase. The detection findings with the performance of the F1-score acquisition system, mAP, have a result of more than 90%, suggesting that the optimization strategy can detect concealed fractures in the pavement image.

Sathvik et al. [39] employed the YOLOv7 model to evaluate road defects using a dataset comprising 289 cases. The study utilized 460×460 images generated through a three-step process involving epoch and weight configurations. The YOLOv7 model was used to annotate and train the pothole image dataset, and the results were evaluated based on recall and accuracy metrics. The model was validated by analyzing a large number of pothole-related images. The results demonstrated an F1 score of 0.51, which increased proportionally with the number of epochs.

Qurishee [71] presented an autonomous pavement condition evaluation employing deep learning, an unmanned aerial vehicle (UAV), and Raspberry Pi tiny computer technologies, which improves road maintenance and renovation management efficiency and cost effectiveness. A low-cost Raspberry Pi smart defect-detecting camera was set up with the trained SSD MobileNet v1, which can be deployed with a UAV for real-time and remote pavement condition inspection. According to preliminary data, the smart pavement detector camera achieves 60% accuracy at 1.2 frames per second on the Raspberry Pi and 96% accuracy at 13.8 frames per second on a CPU-based computer. Hassan et al. [72] employs CNN models that are improved tiny YOLO V3 on UAVs and achieves good results, with precision, sensitivity, and F1 score values better than 85%, but without considering FPS. Sathvik et al. [39] introduce

TABLE 1. Comparison of object detection models deployed on edge device.

Previous studies	[33]	[71]	[34]	[39]	[48]	Proposed System
Model CNN	MobileNet	MobileNetV1	MobileNetV2	YOLOV7	MobileNetV2	iYOLOV7-TPE-SS
Object detection	Eight types of road damages	Pavement	Pothole, longitudinal cracks, alligator cracks	Pothole	Pothole, longitudinal cracks, alligator cracks	Pothole, longitudinal cracks, alligator cracks
Edge Device	Handphone	Raspberry Pi	NVIDIA Jetson Nano	CPU-based computer	NVIDIA Jetson Nano	NVIDIA Jetson Nano NVIDIA AGX Orin
System Performance Properties						
mAP	O	O	X	O	X	O
AR	O	X	O	O	O	O
F1-score	X	X	O	X	X	O
Efficiency Properties						
Inference time	X	O	X	X	X	O
Model size	O	X	X	O	X	O
FPS	X	O	X	X	X	O

pothole detection, an edge computing framework that runs efficient deep neural networks, lowering the latency of earlier techniques. Using transfer learning-based models, an F1 score of 0.64. The YOLO V7 model had an F1 score of 0.51, which improved proportionally (the higher the F1 score, the more efficient the training model) as the number of epochs increased. The YOLO V7 runs on a CPU-based computer, however it does not consider run on edge devices.

Several studies using CNN models deployed on NVIDIA Jetson Nano devices include the SSD-MobileNet model with a batch size of 64 and mAP of 0.22 [48], the MobileNet V2 model, and the SSD Resnet50V1 model [34], [35]. The MobileNet V2 model using hyperparameter settings has a higher mAP of 0.0869 and average recall 0.241 than the SSD Resnet50V1. Yebes et al. [40] created a massive dataset of images with pothole annotations and road scenes from many cities across the world, captured with various cameras, vehicles, and viewpoints under varying environmental circumstances. Their fine-tuned Faster R-CNN Inception v2, Faster R-CNN Resnet101, and Faster R-CNN Inception-Resnet v2 models reached precision 75%, and their CNN model deployed the pothole detector on the Nvidia DrivePX2 platform running at 5–6 frames per second. Based on the findings of this investigation, CNN models were included in the hardware, as indicated in Table 1, although further research is needed to improve system performance (mAP, AR, F1 score) and speed up inference time. Based on previous studies on road damage, several researchers have utilized the CNN model by optimizing its network structure. Specifically, the YOLO model automates the network structure through the introduction of a new module. However, few YOLOv3 models incorporate hyperparameter optimization, with only one study utilizing the Bayesian method.

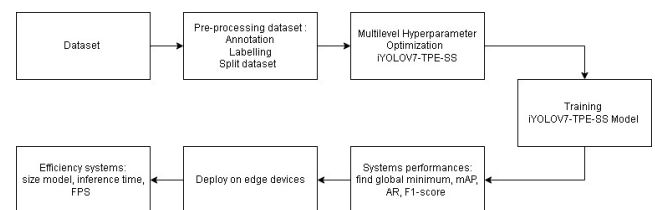
III. PROPOSED SYSTEMS FOR ROAD DAMAGE DETECTION

This road damage detection research consists of multiple stages, including dataset retrieval, dataset pre-processing, iYOLOV7-TPE-SS modeling, calculating system performance, and deploying the iYOLOV7-TPE-SS model on edge devices. The general strategy for automatic road damage

detection using the iYOLOV7-TPE-SS model on edge devices is as follows: data retrieval using multiple sensors, data preprocessing, iYOLOV7-TPE-SS modeling, system performance calculations, deployment of iYOLOV7-TPE-SS model on edge device, and iYOLOV7-TPE-SS model efficiency calculation, specifically inference time and frames per second (FPS). Fig. 2 shows the details of the iYOLOV7-TPE-SS model measurement system performances and deploy on edge devices.

The dataset is then subjected to data preprocessing, which includes annotation and labeling. The dataset has been divided into 70% training and 30% testing datasets. In order to simulate the CNN model, we utilize the iYOLOV7-TPE-SS model using batch size and optimizer settings. The system performance metrics (mAP, AR, and F1) are then calculated. The iYOLOV7-TPE-SS model is deployed on edge devices for real-time and autonomous road damage detection applications, with a focus on model size, inference time, and frames per second (FPS).

Hyperparameter optimization is one of the strategies for improving system performance. Some research uses hyperparameter optimization techniques such as Random Search, Grid Search, Bayesian optimization [53] and Particle Swarm Optimization (PSO) [45]. In addition, there is a tuning hyperparameter between the learning rate and batch size for road damage detection [34], [35]. This study investigates performance enhancement using multilevel hyperparameter optimization using the iYOLOV7-TPE-SS model, which aims to achieve minimal loss in function as well as excellent system performance.

**FIGURE 2.** Stages of modeling the iYOLOV7-TPE-SS model on edge devices.

The best system performance results produced by using the iYOLOV7-TPE-SS model with parameters mAP, recall, and F1 score are then deployed on the edge device. The best system performing iYOLOV7-TPE-SS model is installed on the edge device using inference time and FPS as system efficiency requirements. Because this edge device has limited capacity and memory for direct image analysis. Hence, the iYOLOV7-TPE-SS model's efficiency outcomes are projected to be fast inference time and high FPS.

A. DATASET RETRIEVAL

In this study, dataset retrieval employs datasets obtained directly and integrates a collection of road damage images from the East Java Public Works Office's Directorate of Highways survey results. The road damage tool system [48] was used to collect data on provincial highways in East Java. The image dataset captured yielded 11,176 images used in this study. The datasets have comparable properties to the COCO and KITTI datasets, but only the categories of potholes, longitudinal cracks, and alligator cracks were included. Dataset retrieval employs camera sensors and GPS accelerators coupled with an edge device positioned on the car. The study uses datasets for road defect categorization, including potholes (L00), longitudinal cracks (R02), and alligator cracks (R03) as shown in Fig. 3.

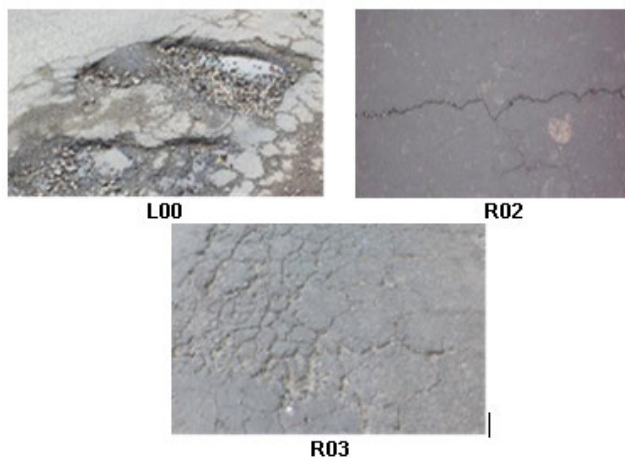


FIGURE 3. Example dataset: potholes (L00); longitudinal cracks (R02); alligator cracks(R03).

B. DATASET PRE-PROCESSING

The next stage in road damage dataset processing is dataset pre-processing, which is required to equalize the image sizes and formats. This dataset processing allows for speedier performance and enhances model performance. Using Windows Utility, the dataset images are converted to a 640×640 pixel image. Annotate the entire dataset to identify it and define its properties based on item classification using the labelling tool.

The annotation method generates a dataset with a bounding box representing the type of road damage classification.

The present study employs four labeling datasets for road defect classification: potholes (L00), longitudinal cracks (R02), alligator cracks (R03), and shadows (00). This dataset labeling approach is appropriate for road damage data and is based on the pavement condition survey form issued by Indonesia's Department of Public Works, Directorate General of Highways. For evaluation reasons, the datasets were randomly split into two groups: 70% for training and 30% for testing.

C. THE IMPROVED YOLOV7 MODEL USING MULTILEVEL HYPERPARAMETER OPTIMIZATION

This study uses CNN models to automatically detect road damage on edge devices. As a result of the limited storage and power consumption required to run the CNN model on edge devices, consideration must be given to its utilization. It is critical to consider the design parameters of the CNN model when utilizing one-stage detectors such as YOLOV7. The primary parts of the recommended strategy are described as follows.

The development of the YOLOV7 model has mostly focused on enhancing the training process by optimizing modules and approaches that can be used to boost accuracy while minimizing inference costs. The YOLOV7 model modifies its model scaling architecture to produce a model scaling method that does not require overly complex rules and has a lower computing cost. It was created by analyzing numerous parameters such as memory access cost, the impact of input/output channel ratio, activation during model scaling, and gradient path. The YOLOV7 model includes numerous model versions designed for specialized applications, including the YOLOV7 model, which is appropriate for use in GPUs on edge computing devices. The YOLOV7 model improves detection accuracy and inference speed.

The YOLOV7 model is a network composed of three primary components: the backbone, the neck, and the head. Its network structure is illustrated in Fig. 4. The YOLOV7 model has a three-stage architecture: an input 640×60 image that has been preprocessed, then the image is routed to the backbone, which extracts multiscale features from the input image, then to the neck, which aggregates features to improve multi-scale object detection, and finally to the head, which produces an output image in the form of a bounding box prediction, class score, and confidence value. The backbone incorporates E-ELAN (Extended Efficient Layer Aggregation Network), CSPDarknet (Cross-Stage Partial Darknet), and SiLU activation modules to achieve efficient feature extraction. The E-ELAN module optimizes gradient propagation, while the CSPDarknet module reduces computational costs by leveraging SiLU activation for improved nonlinearity. The neck includes the PAFPN (Path Aggregation Feature Pyramid Network) module and the SPPF (Spatial Pyramid Pooling-Fast) block function. The PAFPN module enhances multiscale feature integration, and the SPPF block effectively expands the receptive field.

Finally, the head serves as a dedicated detector, generating visual outputs in the form of classification results.

This study investigates system performances enhancement iYOLOV7-TPE-SS model, which it to achieve minimal loss in function as well as excellent system performance. The iYOLOV7-TPE-SS model approach is the YOLOV7 model utilize hyperparameter optimization divided into two stages: the first stage is the TPE optimization method, and the second stage optimizes the hyperparameter using the search space method as shown Fig.3. The multilevel hyperparameter optimization procedure at each stage is as follows:

- 1) **Stage 1: Tree-Structured Parzen Estimator (TPE).** Adjust the YOLOV7 network employing hyperparameters to adjust the learning rate, batch size, and optimizer utilizing TPE hyperparameter optimization. TPE optimization yields system performance metrics such as mAP, recall, F1-score, and accuracy, which are then averaged to obtain the global minimum in the form of the least loss. The next step is to determine how many candidates pass the requirement that the system performance value is above the average

system performance value while experiencing a small loss below the average loss value. These candidate parameters are utilized to enter stage 2.

- 2) **Stage 2: Search Space (SS).** In this step, the Search Space method is used with the TPE result parameter from stage 1. To achieve the best system performance, combine the parameter n with the number step parameter and employ the Search Space approach. The Search Space approach examines the output of various candidates in terms of excellent system performance testing and least loss, resulting in the best performance value. The best candidate in stage 2 is determined based on the highest average system performance, evaluated using metrics such as mAP, AR, F1-score, and accuracy. The average value of system performance is calculated and the best average value of the candidate is selected.

The iYOLOV7-TPE-SS model processes the input data image X by convolution, where w and h denote width and height, and c represents the number of input channels. Define a convolution layer operation that produces m feature maps, as illustrated in Eq.1:

$$Y_i(X_i) = b_i + \sum_{x_i \in X}^n f_{ij} \cdot X_i, \quad (1)$$

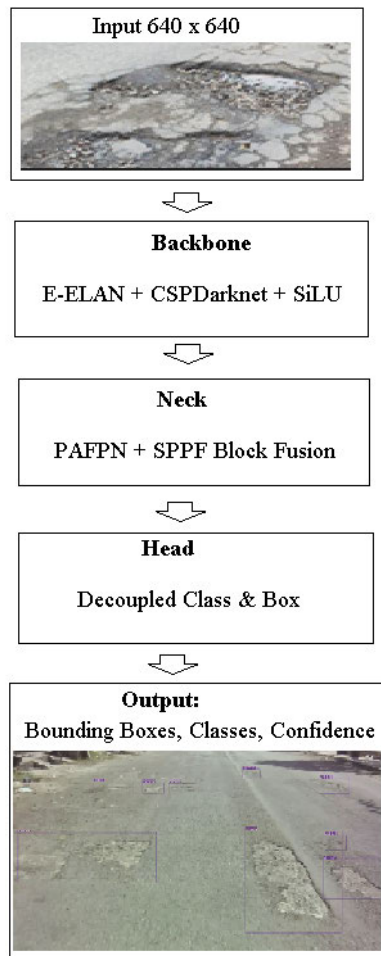


FIGURE 4. The architecture YOLOV7 model for road damage consists of three parts: the backbones, neck and head.

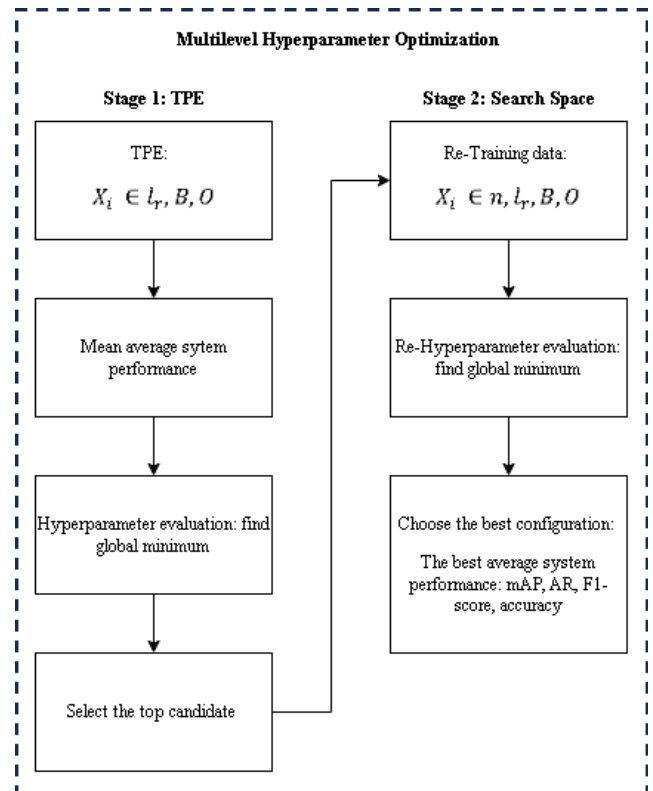


FIGURE 5. The iYOLOV7-TPE-SS model using multilevel hyperparameter optimization.

TABLE 2. The specifications of devices in the measurements.

Specification of	PC	NVIDIA Jetson Nano	NVIDIA AGX Orin
Software	Window 10	Linux	Linux
O/S	Window 10	Ubuntu 18.04/20.04	Ubuntu 20.04
CPU	AMD Ryzen Pro	Quad-core ARM Cortex-A57 MPCore 64 bit ARMv8-A	12 core ARM Cortex-A78AE 64 bit ARMv8
GPU	NVIDIA MX/Quadro	NVIDIA Maxwell 128 CUDA cores.	2048 CUDA Cores 64 Tensor Cores
RAM	4 GB	4 GB	32 GB
Storage	SSD M.2 NVMe, 2 TB	SSD 32 GB	64 GB eMMC 5.1

when Y is the output, b is the bias, $f \in R^{c \times k \times k \times m}$ f is the convolution filter, m is the input channel's feature map, $k \times k$ represents the kernel size for convolution filters f , and i is number of image as input. This study investigates performance improvement using the iYOLOV7-TPE-SS model determining the least loss under various scenarios of optimizer and batch size parameters. Multi-level hyperparameter optimization using iYOLOV7-TPE-SS model yield results in the best system performance and the lowest loss value. The schematic multilevel hyperparameter as shown Fig. 5 and to find global optimum x^* the Eq.2:

$$x^* = \arg \min Y_i(X)_{x \in X} \quad (2)$$

where X are hyperparameters of the iYOLOV7-TPE-SS model and the bayesian optimization hyperparameter using Tree-structured Parzen Estimator (TPE) as shown Eq.3:

$$p(Y_i | X_i) = \begin{cases} G(X_i), & \text{if } Y_i < Y_i^* \\ L(X_i), & \text{if } Y_i \geq Y_i^* \end{cases} \quad (3)$$

TPE optimizes the objective function conditional probability represented as $p(Y_i | X_i)$ is by determining the hyperparameter set. The parameters TPE that are: $G(X_i) = p(X_i | Y_i < Y_i^*)$ hyperparameter distribution that produces an objective value superior to Y_i^* and $L(X_i) = p(X_i | Y_i \geq Y_i^*)$ hyperparameter distribution that produce an objective value lower than Y_i^* . Evaluation of the real objective function by selecting the hyperparameters that perform best on the goal function.

This study employs iYOLOV7-TPE-SS model as the machine learning architecture in a variety of road defect detection applications utilizing edge devices. Hyperparameter optimization using TPE optimization is a recent hyperparameter optimization technique, although it has a restricted number of tuning experiments. After determining the best candidate using TPE optimization, data sampling continues. In robust sampling of the hyperparameter Search Space, we focus solely on the two primary hyperparameters optimizer (O) and batch size (B) to achieve minimal parameter loss using the learning rate and weight functions. The Search Space combination for hyperparameter optimizer and hyperparameter batch size as shown Eq. 4 and Eq. 5:

$$SC_i = SO_i \cdot SB_i, \quad (4)$$

$$SC_i = \{(O, B) | O \in \langle v_i, MS_i, W_i \rangle, B \in \langle 2, 4, 8, 16 \rangle\} \quad (5)$$

where SC_i is Search Space combination, SO_i is Search Space optimizer, and SB_i is Search Space batch size. This study investigates the use of optimizers such as Momentum (v_i), RMSprop (MS_i), and Adam (W_i) in the iYOLOV7-TPE-SS model. By solving the problem of delayed convergence and expediting model training, the Momentum optimizer contributes to the filling of local gaps and improving convergence in the right direction. The iYOLOV7-TPE-SS model parameters are the learning rate and cost function of weights, which yield the Momentum optimizer parameter Eq. 6 and 7:

$$v_i = \beta \cdot v_{(i-1)} + (1 - \beta) \cdot \nabla F(w_i) \quad (6)$$

$$w_{(i-1)} = w_i - \alpha \cdot v_i \quad (7)$$

where i is the parameter at the i -iteration, w is weights, α is the learning rate, β is momentum coefficient (0.9), $\nabla F(w_i)$ is the gradient of the cost function with respect to the parameter at iteration i .

The iYOLOV7-TPE-SS model iteration method employs a learning rate parameter that maintains the average squared gradient for each weight. This approach employs the RMSprop optimizer as one of the optimization strategies, as shown in Eq. 8.

$$MS_{(w,i)} = \left(\beta \cdot MS_{(w,i-1)} + 0.1 \cdot \left(\frac{\partial E}{\partial w_{(i)}} \right) \right)^2 \quad (8)$$

where β is momentum coefficient and MS means square at the i -iteration. The iYOLOV7-TPE-SS model parameters, including learning rate (α) and weights (w), are utilized to calculate gradient, first moment, and second moment with exponential decay rates (β_1, β_2). Eq. 9 represents the calculation results in the configuration of Adam optimizer.

$$W_i = W_{(i-1)} - \alpha \cdot \left(\frac{m_i}{\sqrt{v_i} + \epsilon} \right) \quad (9)$$

where α is the learning rate, m is momentum estimation, v is root mean square estimation, ϵ is constan.

The output $Y_i^*(X_i)$ is created during the simulation of iYOLOV7-TPE-SS model calculations using multilevel hyperparameter optimization scenarios that are a combination Tree Structured Parzen Estimator and Search Space, utilizing Eq. 2 – 10. Optimize the parameter Y_i^* as the output of the CNN models. This study utilizes iYOLOV7-TPE-SS model with the output of road defect classification in

comparison to the real Y_i as shown in Eq. 10:

$$Y_i^* = \min(Y_i(X_i)) \quad (10)$$

Minimize the function using the multilevel hyperparameter optimization using iYOLOV7-TPE-SS model. In the equation, it calculates the system's performance employing the parameters TP, TN, FP, and FN. Given the parameters TP, TN, FP, and FN, calculate the mAP, recall, and F1-Score the values shown in the Eq. 10-12. The evaluation process has calculated systems performance, the value of precision, recall, and F1 score. Precision and recall are obtained from the following Eq. 11 and 12, as shown below [38], [70]:

$$P = \frac{TP}{TP + FP} \quad (11)$$

$$R = \frac{TP}{TP + FN} \quad (12)$$

that are P is precision, R is recall, TP is true positive, false positive is (FP), false negative is (FN).

The total loss is the result of an incorrect forecast, whereas the loss is a numerical figure that indicates the level of error that utilize the iYOLOV7-TPE-SS model. Precision is used as a parameter to determine the level of accuracy between ground truth (real data) and the prediction results of the model. It is just true positives (TP) divided by total detections (TP+FP). To determine how well a positive value is found, recall is employed. It applies true positives (TP) to all ground truth values, which include both true positives and false negatives (FN). The F1 score is an evaluation metric that combines a model's precision and recall scores, as illustrated in the equation that calculates accuracy. The accuracy statistic computes the number of times the iYOLOV7-TPE-SS model predicted accurately throughout the entire dataset [38], [70]:

$$F1score = 2 \cdot \frac{(P \cdot R)}{(P + R)} \quad (13)$$

The system performance of the iYOLOV7-TPE-SS model has been evaluated using a metric known as mean average accuracy (mAP), which takes into account the average recall (AR), F1-score, and inference time. The inference time has the inverse of the FPS, and both are the major measures of model detection rate. Frames per second (FPS) refers to the number of frames (images) transmitted per second, whereas inference time refers to the time taken by the model to process an image.

In this study, which is the iYOLOV7-TPE-SS model, was employed to increase system performance, including mAP, AR, and computation time. The batch size determines the amount of training used in a single iteration of the iYOLOV7-TPE-SS model. This has the potential to improve system performance as well as generalize the iYOLOV7-TPE-SS model. CNN model generalization refers to the ability of the iYOLOV7-TPE-SS model to function better when given different inputs than the previous one, resulting in an iYOLOV7-TPE-SS model that works well and has

strong system performance. Larger batch sizes can enhance generalization, allowing the iYOLOV7-TPE-SS model to average more weight while minimizing weight variance.

The best system performance results obtained utilizing the iYOLOV7-TPE-SS model with the parameters mAP, recall, and F1 score are then deployed on the edge devices. In this study, we employ a PC to utilize the iYOLOV7-TPE-SS model to determine the optimal system performance parameters. Furthermore, for evaluating the efficiency of the iYOLOV7-TPE-SS model, it is implemented on edge devices that include embedded edge device 1, the NVIDIA Jetson Nano, and embedded device 2, the NVIDIA AGX Orin. The research results of the iYOLOV7-TPE-SS model deployment are utilized as a guide for practical road damage identification.

The specification of devices in the measurement in Table 2. The best-performing iYOLOV7-TPE-SS model is deployed on the edge device based on inference time and FPS as system efficiency parameters. Inference time is the time in seconds that the iYOLOV7-TPE-SS model takes to detect road damage. The parameter represents the number of road damage images that can be detected in one second, represented in frames per second. The iYOLOV7-TPE-SS model's efficiency outcomes will be expected to be fast inference time and high FPS because this edge device has limited capacity and memory to analyze images directly.

IV. PERFORMANCE MEASUREMENT AND ANALYSIS

The collection of the dataset utilized iYOLOV7-TPE-SS modeling, which was executed on a PC device with a 4 GB RAM processor and an operating system such as Table 2. The iYOLOV7-TPE-SS model produces system performance measurements such as mAP, AR, and F1-score. In a road damage detection study, the iYOLOV7-TPE-SS model employs multilevel hyperparameter optimization.

The iYOLOV7-TPE-SS model is converted to TF Lite and then deployed on an embedded edge devices using NVIDIA Jetson Nano and NVIDIA AGX Orin. Deploying the edge devices based on efficiency, which is expressed by characteristics like inference time, frames per second, and CNN model size. This expresses the capacity to process the identification of road damage items using the iYOLOV7-TPE-SS model in terms of inference time. The iYOLOV7-TPE-SS model results are deployed on edge devices to assess its performance on the device. The goal has been to deploy the iYOLOV7-TPE-SS model on an edge device in order to create a road damage detection device that can operate autonomously and in real time.

A. SYSTEM PERFORMANCE

This study employs the iYOLOV7-TPE-SS model, focusing on performance measurement and result analysis, particularly in the discussion of evaluation metrics, test methodologies, and results interpretation. In data discussion, model performance is evaluated by identifying how well the model performs on a specific task, and generalization is validated

by assessing whether the model works well on previously unseen data or test data. In addition, this study compares the iYOLOV7-TPE-SS model's performance against that of the baseline or other models.

The iYOLOV7-TPE-SS model is a hyperparameter optimization multilevel model that combines stage one hyperparameter optimization using TPE and stage two using SS optimization. The iYOLOV7-TPE-SS model receives road damage data as input and outputs system performance and loss numbers at each stage. The input image is distributed to the advanced backbone, neck and head of the iYOLOV7-TPE-SS model and compared to the neck, dataset image to determine the TP, TN, FP, and FN parameters. In stage 1, the performance evaluation analysis on system performance has the form of average system performance (mAP, recall, and F1 score) on the objective function as well as the lowest loss value. Then, identify multiple hyperparameter candidates that perform well on objective functions with low loss values and good system performance. Furthermore, stage 2 data processing in the Search-Space model has better system performance and the lowest loss.

Furthermore, we examined and evaluated the experimental findings of the system performance system, which are total loss and average system performance (mAP, recall, and F1 score). Based on the results findings of stage 1, hyperparameter optimization using TPE is shown in Table 3. The iYOLOV7-TPE-SS model uses TPE computation results to calculate average loss and mAP, which are then used as calculation limitations. The loss calculation limit value is lower than the average loss calculation limit, whereas the mAP calculation limit value is higher than the average mAP value. Based on the computation limit values of loss and mAP, there are three good performance possibilities, including YOLOV7 with these hyperparameters: (1) Adam, batch size 4 (2). Adam, batch size 8 (3). Adam; batch size: 16.

Based on the findings of the TPE hyperparameter optimization with multiple candidates, the optimal results are used to determine the optimal results using the stage 2 procedure: hyperparameter search space optimization. According to the TPE optimization findings in Table 3, hyperparameter optimization employs a combination of search space and the Adam optimizer, as well as batch sizes of 4, 8, and 16. Fig. 4 summarizes the outcomes of multilevel hyperparameter optimization using TPE and search space, which is the result of the iYOLOV7-TPE-SS model.

Simulation of the iYOLOV7-TPE-SS model findings demonstrates that the results of system performance metrics mAP, AR, and F1-Score. The iYOLOV7-TPE-SS model has the best system performance, which employs a learning rate of 0.001, a number of steps of 100000, a batch size of 16 and Adam as the optimizer. The system performance has a results precision value of 0.986 and an F-score of 0.978. The road damage recognition results reveal that the iYOLOV7-TPE-SS model can detect the bounding box detection image with the intersection between the test image and the ground via IoU 0.5, with a mAP@0.5 value of 0.988. Due to the relatively

TABLE 3. Model TPE-based hyperparameter optimization yields loss Values and average system performance.

No.	Loss	mAP	Hyperparameter
1	0.0807	0.0293	RMSprop, 16
2	0.0778	0.0496	Momentum, 16
3	0.0637	0.1709	Adam, 4
4	0.0660	0.1700	Adam, 16
5	0.0818	0.0280	RMSprop, 4
6	0.0799	0.0583	Momentum, 16
7	0.0819	0.0139	RMSprop, 8
8	0.0777	0.0467	RMSprop, 16
9	0.0780	0.0353	RMSprop, 16
10	0.0658	0.1817	Adam, 8
11	0.0643	0.1661	Adam, 4
12	0.0639	0.1693	Adam, 4
13	0.0632	0.1806	Adam, 4
14	0.0644	0.1734	Adam, 4
15	0.0632	0.1806	Adam, 4
16	0.0634	0.1781	Adam, 4
17	0.0745	0.0755	Momentum, 4
18	0.0639	0.1688	Adam, 4
19	0.0660	0.1913	Adam, 8
20	0.0640	0.1551	Adam, 4
21	0.0746	0.0666	Momentum, 4
22	0.0637	0.1814	Adam, 4
23	0.0642	0.1712	Adam, 4
24	0.0642	0.176	Adam, 4
25	0.0640	0.1661	Adam, 4
26	0.0651	0.1928	Adam, 8
27	0.0637	0.1832	Adam, 4
28	0.0635	0.1685	Adam, 4
29	0.0736	0.1341	Momentum, 4
30	0.0659	0.1967	Adam, 8
Average	0.0688	0.1503	

high mAP value, the iYOLOV7-TPE-SS model can recognize the image with an IoU of 0.5-0.95 of 0.806. Changing the derivative with Adam's second moment on the weight of each iteration in the iYOLOV7-TPE-SS model can increase the system performance.

When comparing the system performance of the iYOLOV7-TPE-SS model and previous research results, the iYOLOV7-TPE-SS model performs the latter in terms of identifying road damage. Based on the best system performance Fig. 6, the iYOLOV7-TPE-SS model has precision, mAP@0.5, AR, and F1-score values near 0.98. The iYOLOV7-TPE-SS model, which demonstrates the best system performance, is deployed on an edge device using the NVIDIA Jetson Nano. The detection results are presented in Fig. 7.

The findings of this study increased the system performance value of earlier researchers [33], [34], and [48], as indicated in Table 1. This research has road damage detection applications: potholes, longitudinal cracks, and alligator cracks on the edge device, namely Jetson Nano, but still below researcher [40], who employed the edge device, namely the Nvidia DrivePX2, for pothole identification even though it has a larger processor. It can apply a two-stage CNN model and provide higher performance than this research using a single-stage CNN model.

However, one disadvantage of image processing-based road inspection approaches is that there is no common dataset for comparing findings; each study evaluates the proposed method using its own road images dataset [33]. Some studies offer road damage datasets with varying levels of resolution, depending on the camera used and how the images were captured. As a result, this dataset has an impact on system performance through data processing and CNN modeling. It is challenging to detect road damage precisely and automatically in real time utilizing the CNN deployed on an edge device.

When applying the iYOLOV7-TPE-SS model on edge devices for road damage identification, there is a critical relationship between mAP, AR, F1-score, and frames per second (FPS) while capturing images with a moving sensor. Multilevel hyperparameter optimization using TPE-SS improves system performance. As shown in Fig. 6, a high mAP indicates that the model can accurately recognize a wide range of road damage types, including longitudinal cracks, alligator cracks, and potholes. High accuracy and AR values ensure that the iYOLOv7-TPE-SS model can detect as many damage cases as possible. A balanced F1-score reflects an optimal trade-off between precision and recall, which is crucial to minimizing false positives and false negatives. However, on edge devices with limited computational resources, complex models may reduce FPS, leading to delays in image processing, especially when the vehicle is moving at high speeds. If the FPS is too low, issues such as blurring or frame loss may occur, reducing detection accuracy. Therefore, a lightweight YOLOv7-tiny model is necessary to achieve a balance between accuracy and speed in object detection. This ensures real-time detection without compromising system performance on edge devices used in moving vehicles. Consequently, this study compares the performance and efficiency of the YOLOv7-tiny model with the YOLOv7-TPE-SS model.

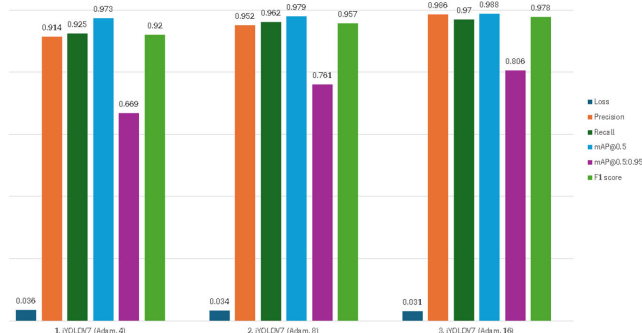


FIGURE 6. The result system performances of iYOLOV7-TPE-SS model.

B. EFFICIENCY MEASUREMENT

The CNN model must be deployed on edge devices to automatically detect road damage, as stated in the research objectives. In this study, the effects of optimization

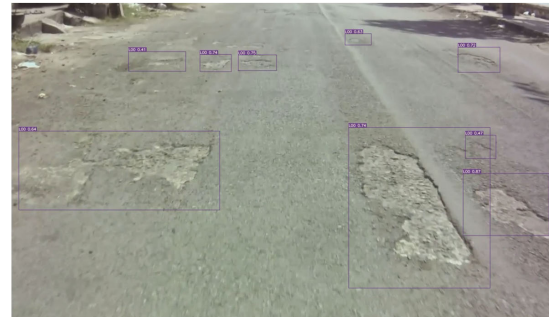


FIGURE 7. Prediction results iYOLOV7-TPE-SS model.

hyperparameters, which is iYOLOV7-TPE-SS model settings, are simulated to determine system efficiency as frames per second (FPS), inference time, and size model. Employing the iYOLOV7-TPE-SS model, this research adjusts hyperparameters such as batch size and builds an optimizer to improve system performance. Average frame rate (FPS) and average inference time are system performance measures. The outcomes of installing the iYOLOV7-TPE-SS model on the edge device, including variations in hyperparameter optimization.

The performance of the iYOLOV7-TPE-SS model on embedded edge devices has direct field-testing results, as shown in Table 4. The embedded edge devices using the NVIDIA Jetson Nano (embedded edge device 1) and the NVIDIA AGX Orin (embedded edge device 2). This indicates that altering the batch size and optimizer hyperparameters can improve system performance (mAP, recall, and F1 score) while also reducing inference time. Inference time for smaller values and model sizes shows that the iYOLOV7-TPE-SS model is more effective at recognizing road damage.

However, the deployment findings on edge device 1 as NVIDIA jetson Nano show that the model iYOLOV7-TPE-SS showed inference time around 0.135 s and FPS 7.470. When deployed on edge device2 as the NVIDIA AGX Orin, the YOLOV7-TPE-SS model has a faster inference time of 0.014 and an FPS of 67.034 as shown Table 4. According to the edge device characteristics in Table 2, embedded edge device 2 has bigger CPU, GPU, and RAM capacities than embedded edge device 1. This is used to process road damage objects faster, which affects inference time and FPS.

C. SCALABILITY MEASUREMENT ON EDGE DEVICE

Real-time road damage detection necessitates a CNN model that can be deployed on edge devices. CNN models require optimal system performance as well as excellent efficiency in order to be implemented on edge devices, necessitating a trade-off between system performance and system efficiency. Because of its small structure, the YOLOV7 tiny model's system performance and efficiency were calculated in this study [73]. Scalability has been evaluated in this study through the comparison of the iYOLOV7-TPE-SS model versus the YOLOV7 tiny model.

TABLE 4. Comparison of efficiency performances on edge devices.

The CNN model	NVIDIA Jetson Nano			NVIDIA AGX Orin		
	Model size (MB)	FPS	Inference time (s)	Model size (MB)	FPS	Inference time (s)
YOLOV7 [39]	12	7.180	0.140	12	50.594	0.019
YOLOV7-tiny [73]	12	7.129	0.141	12	49.742	0.020
YOLOV7-TPE-SS	12	7.470	0.135	12	67.034	0.014

The iYOLOV7-TPE-SS model's real-time detection capabilities provide precise classification results. Real-time road damage detection is achieved using a camera sensor connected to an edge device mounted on the vehicle. The road damage detection process involves capturing images directly from a moving vehicle equipped with a camera sensor and the iYOLOV7-TPE-SS model deployed on edge devices.

Real-time road damage identification faces two main challenges: balancing system performance and system efficiency. In real-time road damage detection using an edge device mounted on a vehicle, the task is complicated by the small size of road damage objects, such as cracks, and the detection environment, which typically consists of gray asphalt. Additionally, the color of road damage objects (often black) is nearly identical to the background, making detection more difficult. Another critical factor is the use of the iYOLOV7-TPE-SS model on a moving edge device, which requires careful consideration of system performance and efficiency, particularly in terms of FPS (frames per second) and inference time for the object recognition process, as illustrated in Fig. 1. To address these challenges, achieving a trade-off between accuracy and efficiency necessitates a model that is compact yet maintains strong system performance. This research explores scalability by comparing the size and performance of the iYOLOV7-TPE-SS model with the YOLOV7-tiny model.

Furthermore, the system performance of the iYOLOV7-TPE-SS model is compared to the YOLOV7-tiny model as a scalability measurement. In general, the iYOLOV7-TPE-SS and the YOLOV7-tiny structures are similar to a one-stage detector; however, the YOLOV7 model has a more complicated E-ELAN (Extended Efficient Layer Aggregation Network) backbone design than YOLOV7-tiny, which has a simpler backbone. The complex YOLOV7 neck architecture utilizes modules such as PANet (Path Aggregation Network), while the simpler YOLOV7-tiny uses a PANet-like module.

Scalability testing of the system performance and efficiency of iYOLOV7-TPE-SS and YOLOV7-tiny models on edge devices. This additionally enables understanding the hardware limits, but it also allows for the optimization hyperparameter in order to achieve optimum system performance and efficiency on edge devices. This is essential to the successful implementation of the iYOLOV7-TPE-SS model for real-time road defect identification employing edge devices, particularly those that require high efficiency and have limited resources, memory, or computational capacity. This scalability test determines the CNN model's detection

speed in capturing the detection object in one second, which is a system inference time parameter, frames per second (FPS).

The YOLOV7, YOLOV7 tiny and iYOLOV7-TPE-SS models have a faster inference time than the MobileNetV2 model. The fastest inference time result of the iYOLOV7-TPE-SS and YOLOV7-tiny models is 0.135 seconds. This is due to the architecture of the iYOLOV7 and YOLOV7-tiny models, which have a simpler architecture than the MobileNetV2 model. According to the inference time MobileNetV2 is 0.28 seconds. The iYOLOV7-TPE-SS and the YOLOV7-tiny models have inference time results of 0.135 seconds and 0.140 seconds, as shown in Table 4.

However, the YOLOV7, YOLOV7 tiny and iYOLOV7-TPE-SS models are faster at inference than MobileNetV2 models. This demonstrates that the YOLOV7 model has a smaller structure, resulting in higher system efficiency and faster inference time, which makes it suitable for deployment on edge devices. The iYOLOV7-TPE-SS and YOLOV7-tiny models have inference times of 0.141 seconds and 0.135 seconds, respectively, indicating inference conclusions that are almost identical as shown Table 4. This demonstrates that the iYOLOV7-TPE-SS model has a scalability function similar to the YOLOV7-tiny model, in that it can be deployed on edge devices.

Based on the results of this study to deploy the iYOLOV7-TPE-SS and YOLOV7-tiny models on edge devices, Table 3 and Table 4 show that the FPS and inference time are overall good. However, the calculation of system efficiency cannot be done individually; instead, it must be combined with the findings of the system performance calculation to yield the optimum results. According to the results of the iYOLOV7-TPE-SS model with batch size of 16 and the Adam optimizer has the best system performance. The system efficiency values are an inference time of 0.135 seconds, a FPS of 7.12, and a model size of 12 MB.

D. COMPARISON OF CNN MODELS ON EDGE DEVICE

In this study, CNN models such as the MobileNetV2 model, the YOLOV7 model, the YOLOV7-tiny model, and the iYOLOV7-TPE-SS model. The iYOLOV7-TPE-SS model performs excellently in scalability experiments on the NVIDIA Jetson (embedded edge device 1) and NVIDIA AGX Orin (embedded edge device 2) hardware. The iYOLOV7-TPE-SS model developed is deployed on an embedded edge devices, and it is utilized to detect road damage in real time.

According to the aims of this research, the CNN model is installed on the edge device shown in Table 1. This

TABLE 5. The comparison of performance measurement between our proposed iYOLOV7-TPE-SS and several existing models.

The CNN Models	MobileNet V2 [33] [34] [48]	YOLOV7 [39]	YOLOV7-tiny [73]	Our proposed: iYOLOV7-TPE-SS
Parameters	Systems performances			
Loss	0.320	0.022	0.032	0.031
mAP (@IoU 0.5-0.95)	0.298	0.438	0.143	0.806
AR	0.476	0.429	0.450	0.970
F1-score (@IoU 0.5-0.95)	0.322	0.433	0.462	0.880
F1-score (@IoU 0.5)	0.481	0.272	0.462	0.978
Parameters	Efficiency properties edge device 1			
Model size (MB)	3.700	12	12	12
FPS	3.570	7.180	7.129	7.470
Inference time (s)	0.280	0.140	0.141	0.135
Parameters	Efficiency properties edge device 2			
Model size (MB)	3.700	12	12	12
FPS	13.960	50.594	49.742	67.034
Inference time (s)	0.041	0.019	0.020	0.014

study employs the iYOLOV7-TPE-SS model to compare earlier studies utilizing the MobileNetV2 and Faster R-CNN ResNet101 models deployed on edge devices such as HP and NVIDIA Jetson Nano. The system performance measures used to compare CNN models on edge devices include mAP, recall, F1-score, and system efficiency, which includes inference time, FPS, and model size.

Table 5 presents a comparison of system performance and efficiency across various existing models. Researchers employing the MobileNet V2 model [33], [34], [48] with the tuning hyperparameter achieved lower recall and precision compared to the mAP results shown in Table 5, but higher recall and precision than those reported by researchers [34], [48]. Table 5 also indicates that the MobileNet V2 model demonstrates higher inference time and FPS (frames per second) compared to the results from researchers [33], [34], [48]. The YOLOV7 model [39] achieves a significantly higher F1 score, surpassing the results in Table 5 by 0.1. According to researcher [73], the YOLOV7-tiny model with Mish and SiLU activation function optimization yields similar results to this study in terms of mAP@0.5, while mAP@0.5:0.95 is higher than the findings reported by researcher [73] in Table 5. Additionally, this study includes calculations of inference time and FPS on two devices. The iYOLOV7-TPE-SS model, applied in this study with multilevel hyperparameter optimization, outperforms the baseline models [33], [34], [39], [48], [73].

This study utilizes a dataset from previous research, as reviewed in Table 1. The most common type of road damage classification and the largest amount of data were employed by researchers [33]. While this study also focuses on road damage classification, the dataset used here exceeds the size of those used by previous researchers [34], [39], [48]. The YOLOv7-tiny model is referenced by the researcher [73]; however, it is not applied for road damage classification. According to researcher [33], there is no standardized dataset for road damage, meaning each dataset is unique and presents distinct challenges. Based on this research, Table 5 demonstrates that the datasets used remain valid, as the system performance metrics (mAP, AR, F1-score) achieved

with the baseline models namely, the MobileNet V2 model, the YOLOV7 model, and the YOLOV7-tiny model are comparable to those of previous studies. However, the results of this study using the iYOLOV7-TPE-SS model reveal significantly improved system performance (mAP, AR, F1-score) compared to the baseline models [33], [34], [39], [48], [73].

Furthermore, deploying the iYOLOV7 model on the Jetson Nano edge device results in a greater FPS than researcher [40] employing model Faster R-CNN Resnet101 on edge devices with more powerful CPUs. Overall, the iYOLOV7-TPE-SS model may boost FPS and reduce inference time, allowing it to detect road damage items quickly on edge devices with limited storage and power, such as the NVIDIA Jetson Nano.

Furthermore, deploying the iYOLOV7-TPE-SS model on the NVIDIA Jetson Nano edge device results in a greater FPS than researcher [40] employing model Faster R-CNN Resnet101 on edge devices with more powerful CPUs. Overall, the iYOLOV7-TPE-SS model may boost FPS and reduce inference time, allowing it to detect road damage items quickly on edge devices with limited storage and power, such as the NVIDIA Jetson Nano.

The challenges of implementing a road damage detection device with an edge device so that it can recognize road damage automatically and in real time. According to Table 5, this research uses its own dataset versus a centralized dataset, which has its own set of issues [33]. The findings of this iYOLOV7-TPE-SS model investigation show that the system performance and efficiency outperform the researcher [33]. The addition of multilayer hyperparameter optimization demonstrates that system performance can be improved by combining hyperparameter optimization.

The iYOLOV7-TPE-SS model for road damage detection shows promising results, but there is still room for improvement. System performance can be further enhanced by integrating additional modules into the YOLOv7 model architecture, such as New Residual Blocks, Spatial Pyramid Pooling (SPP), and modifications to the Path Aggregation Network. Additionally, employing hyperparameter optimization techniques such as grid search, random search, Bayesian

optimization, and TPE can improve system performance while minimizing loss. Combining the YOLOv7 model's network architecture with hyperparameter optimization ultimately leads to more robust system performance.

Moreover, the iYOLOV7-TPE-SS model demonstrates better scalability and faster inference times compared to the previous CNN model. This indicates that the iYOLOV7-TPE-SS model can detect objects more quickly, making it better suited for deployment on edge devices. The model's efficiency continues to improve in terms of inference time and frames per second (FPS), which is critical for its deployment on edge devices like the NVIDIA Jetson Nano. The edge device was used to collect road damage image data while in motion. To further optimize the model, improvements can be made through techniques such as quantization, pruning, and other approaches. However, it is essential to balance the trade-off between system performance and efficiency [74], [75], [76], [77].

The rapid development of deep learning and AI utilizing CNN models has resulted in the creation of CNN models for deployment on edge devices, known as Edge AI. In the application of rapid development of AI models, particularly CNN models, the models grow in size and complexity; nevertheless, edge devices have limited resources and have substantial issues executing these sophisticated models. The crucial thing to consider is deploying the CNN model on an edge device with limited storage and power. This is given in terms of inference time and detection time in frames per second. According to Tables 5, there are still various options to improve inference time and FPS, such as using accelerators, adding pruning techniques, post-training quantization, and others [74], [75], [76], [77].

V. CONCLUSION

This study's iYOLOV7-TPE-SS model investigates system performance as mAP, recall, and F1-score improvement utilizing multilevel hyperparameter optimization with a Tree-Structured Parzen Estimator and Space search. Based on the results, the iYOLOV7-TPE-SS model that uses hyperparameter Adam and batch size 16, provides the highest system performance and the lowest loss. The results of the system performances of the iYOLOV7-TPE-SS model are total loss of 0.031, precision of 0.986, recall of 0.970, mAP @0.5 of 0.988, mAP @0.5:0.95 of 0.806, and F1-score of 0.978. The iYOLOV7-TPE-SS model is suitable for real-time road damage detection on edge devices due to its high scalability and fast inference time, comparable to YOLOV7-tiny as a lightweight model (0.135 sec and 7.47 FPS on the NVIDIA Jetson Nano). Although the embedded edge NVIDIA AGX Orin device may produce more FPS of 67.034 and has a faster inference time of 0.014 seconds. Our future work in road damage detection development will leverage CNN models installed on edge devices, which enable mobile and real-time data road damage collection. Several recommendations are offered for further research or implementation. First, system performance can be

enhanced by integrating various modules into the YOLOV7 framework, such as New Residual Blocks, Spatial Pyramid Pooling (SPP), and modifications to the Path Aggregation Network. Additionally, optimizing the YOLOV7 model's hyperparameters using techniques like grid search, random search, Bayesian optimization, and TPE can further improve performance. Second, the implementation of CNN models on edge devices for road detection systems can be upgraded by balancing system performance and efficiency through methods such as quantization, pruning, and other optimization techniques. To improve implementation efficiency, strategies like post-training quantization and converting TensorFlow Lite models to TensorRT can be employed to reduce inference time. Finally, the edge device model can be utilized for real-time road defect detection using UAV (Unmanned Aerial Vehicle) devices.

ACKNOWLEDGMENT

The authors appreciate Politeknik Elektronika Negeri Surabaya (PENS) and the Institut Teknologi Sepuluh Nopember (ITS) for providing laboratory equipment and facilities for this study. They are grateful Prof. Ir Amang Sudarsono for providing advice to improve the writing of the study.

REFERENCES

- [1] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, and D. O. Wu, "Edge computing in industrial Internet of Things: Architecture, advances and challenges," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 4, pp. 2462–2488, 4th Quart., 2020.
- [2] H. Mahmudah, A. Musyafa, A. S. Aisjah, S. Arifin, and C. A. Prastyanto, "Digital twin: Challenge road damage detection on edge device," *Chem. Eng. Trans.*, vol. 109, pp. 601–606, 2024.
- [3] L. Zhao, Y. Wu, X. Luo, and Y. Yuan, "Automatic defect detection of pavement diseases," *Remote Sens.*, vol. 14, no. 19, p. 4836, Sep. 2022.
- [4] I. Katsamenis, N. Bakalos, E. Protopapadakis, E. E. Karolou, G. Kopsiaftis, and A. Voulodimos, "Real time road defect monitoring from UAV visual data sources," in *Proc. 16th Int. Conf. Pervasive Technol. Rel. Assistive Environ.*, Jul. 2023, pp. 603–609.
- [5] M. Rathee, B. Bačí, and M. Doborjeh, "Automated road defect and anomaly detection for traffic safety: A systematic review," *Sensors*, vol. 23, no. 12, p. 5656, Jun. 2023.
- [6] B. Setiadi, Supriyono, and D. Purwanto, "Surface distress index updates to improve crack damage evaluation," in *Proc. 11th Asia-Pacific Transp. Environ. Conf. (APTE)*, 2019, pp. 274–281.
- [7] W. Nur, B. S. Subagio, and E. S. Hariyadi, "Relationship between the pavement condition index (PCI), present serviceability index (PSI), and surface distress index on soekarno hatta road, Bandung," *Jurnal Teknik Sipil*, vol. 26, no. 2, pp. 111–120, Aug. 2019.
- [8] M. Surbakti, S. Samsuri, R. Anas, and A. P. M. Tarigan, "Evaluation of road maintenance program based on international roughness index (IRI) and surface distress index (SDI)," in *ICCOEE: Proc. 6th Int. Conf. Civil, Offshore Environ. Eng. (ICCOEE)*, Dec. 2020, pp. 764–771.
- [9] M. Melyar, M. Isya, and S. M. Saleh, "Pavement condition assessment using SDI and PCI method on geumpang road–west Aceh boundary," *IOP Conf. Ser., Mater. Sci. Eng.*, vol. 1087, no. 1, Feb. 2021, Art. no. 012041.
- [10] P. Paikun, E. Suminar, A. Irawan, and S. Bahri, "Determining road handling according to the level of damage using surface distress index (SDI) method," *Astonjadro*, vol. 10, no. 1, pp. 135–149, Mar. 2021.
- [11] K. De Zoysa, C. Keppitiyagama, G. P. Seneviratne, and W. W. A. T. Shihan, "A public transport system based sensor network for road surface condition monitoring," in *Proc. Workshop Netw. Syst. Developing Regions*, Aug. 2007, pp. 1–6.

- [12] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, "The pothole patrol: Using a mobile sensor network for road surface monitoring," in *Proc. 6th Int. Conf. Mobile Syst., Appl., services*, Jun. 2008, pp. 29–39.
- [13] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, "Real time pothole detection using Android smartphones with accelerometers," in *Proc. Int. Conf. Distrib. Comput. Sensor Syst. Workshops (DCOSS)*, Jun. 2011, pp. 1–6.
- [14] S. Rana and Asaduzzaman, "Vibration based pavement roughness monitoring system using vehicle dynamics and smartphone with estimated vehicle parameters," *Results Eng.*, vol. 12, Dec. 2021, Art. no. 100294.
- [15] A. Martinelli, M. Meocci, M. Dolfi, V. Branzi, S. Morosi, F. Argenti, L. Berzi, and T. Consumi, "Road surface anomaly assessment using low-cost accelerometers: A machine learning approach," *Sensors*, vol. 22, no. 10, p. 3788, May 2022.
- [16] A. Basavaraju, J. Du, F. Zhou, and J. Ji, "A machine learning approach to road surface anomaly assessment using smartphone sensors," *IEEE Sensors J.*, vol. 20, no. 5, pp. 2635–2647, Mar. 2020.
- [17] T. Shen, G. Schamp, and M. Haddad, "Stereo vision based road surface preview," in *Proc. 17th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2014, pp. 1843–1849.
- [18] N. Z. Nuzula, R. W. Sudibyo, H. Mahmudah, and N. Sa'Adah, "Implementation of LiDAR array system for damaged road characteristic identification on moving vehicle," in *Proc. Int. Electron. Symp. (IES)*, Aug. 2023, pp. 244–249.
- [19] M. R. De Blasii, A. Di Benedetto, M. Fiani, and M. Garozzo, "Assessing of the road pavement roughness by means of LiDAR technology," *Coatings*, vol. 11, no. 1, p. 17, Dec. 2020.
- [20] Z. Feng, A. El Issaoui, M. Lehtomäki, M. Ingman, H. Kaartinen, A. Kukko, J. Savela, H. Hyyppä, and J. Hyyppä, "Pavement distress detection using terrestrial laser scanning point clouds—accuracy evaluation and algorithm comparison," *ISPRS Open J. Photogramm. Remote Sens.*, vol. 3, Jan. 2022, Art. no. 100010.
- [21] G. M. Jog, C. Koch, M. Golparvar-Fard, and I. Brilakis, "Pothole properties measurement through visual 2D recognition and 3D reconstruction," *Comput. Civil Eng.*, vol. 2012, pp. 553–560, Jun. 2012.
- [22] E. Buza, S. Omanovic, and A. Huseinovic, "Pothole detection with image processing and spectral clustering," in *Proc. 2nd Int. Conf. Inf. Technol. Comput. Netw.*, vol. 810, Oman, 2013, pp. 48–53.
- [23] L. Huidrom, L. K. Das, and S. K. Sud, "Method for automated assessment of potholes, cracks and patches from road surface video clips," *Proc.-Social Behav. Sci.*, vol. 104, pp. 312–321, Dec. 2013.
- [24] M. B. S. G. Naik and V. Nirmalrani, "Detecting potholes using image processing techniques and real-world footage," in *Proc. Cognit. Informat. Soft Comput. (CISC)*, Jan. 2021, pp. 893–902.
- [25] R. Sharma, K. Singh, and L. Chand, "Analysis of image processing techniques for road anomalies detection," *Int. J. Emerg. Res. Manage. Technol.*, vol. 5, 2016.
- [26] M. Gao, X. Wang, S. Zhu, and P. Guan, "Detection and segmentation of cement concrete pavement pothole based on image processing technology," *Math. Problems Eng.*, vol. 2020, pp. 1–13, Jan. 2020.
- [27] H. Fang and N. He, "Detection method of cracks in expressway asphalt pavement based on digital image processing technology," *Appl. Sci.*, vol. 13, no. 22, p. 12270, Nov. 2023.
- [28] M. H. Yousaf, K. Azhar, F. Murtaza, and F. Hussain, "Visual analysis of asphalt pavement for detection and localization of potholes," *Adv. Eng. Informat.*, vol. 38, pp. 527–537, Oct. 2018.
- [29] N.-D. Hoang, "An artificial intelligence method for asphalt pavement pothole detection using least squares support vector machine and neural network with steerable filter-based feature extraction," *Adv. Civil Eng.*, vol. 2018, no. 1, Jan. 2018, Art. no. 7419058.
- [30] H. Song, K. Baek, and Y.-C. Byun, "Pothole detection using machine learning," *Adv. Sci. Technol.*, vol. 2018, pp. 151–155, Feb. 2018.
- [31] N.-D. Hoang, T.-C. Huynh, and V.-D. Tran, "Computer vision-based patched and unpatched pothole classification using machine learning approach optimized by forensic-based investigation Metaheuristic," *Complexity*, vol. 2021, no. 1, Jan. 2021, Art. no. 3511375.
- [32] C. Chun and S.-K. Ryu, "Road surface damage detection using fully convolutional neural networks and semi-supervised learning," *Sensors*, vol. 19, no. 24, p. 5501, Dec. 2019.
- [33] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiya, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 33, no. 12, pp. 1127–1141, 2018.
- [34] Z. S. Hernanda, H. Mahmudah, and R. W. Sudibyo, "CNN-based hyperparameter optimization approach for road pothole and crack detection systems," in *Proc. IEEE World AI IoT Congr. (AIoT)*, 2022, pp. 538–543.
- [35] A. C. Aqsa, H. Mahmudah, and R. W. Sudibyo, "Detection and classification of road damage using CNN with hyperparameter optimization," in *Proc. 6th Int. Conf. Informat. Comput. Sci. (ICICoS)*, Sep. 2022, pp. 101–104.
- [36] R. W. Sudibyo and A. Alimudin, "Design and implementation of artificial intelligence-based real-time pothole detection for IoT smart infrastructure," *Inf., Jurnal Ilmiah Bidang Teknologi Informasi Dan Komunikasi*, vol. 8, no. 2, pp. 125–131, Jun. 2023.
- [37] A. Zhang, K. C. P. Wang, Y. Fei, Y. Liu, S. Tao, C. Chen, J. Q. Li, and B. Li, "Deep learning-based fully automated pavement crack detection on 3D asphalt surfaces with an improved CrackNet," *J. Comput. Civil Eng.*, vol. 32, no. 5, Sep. 2018, Art. no. 04018041.
- [38] Z. Liu, X. Gu, H. Yang, L. Wang, Y. Chen, and D. Wang, "Novel YOLOv3 model with structure and hyperparameter optimization for detection of pavement concealed cracks in GPR images," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 22258–22268, Nov. 2022.
- [39] M. Sathvik, G. Saranya, and S. Karpagaselvi, "An intelligent convolutional neural network based potholes detection using YOLO-V7," in *Proc. Int. Conf. Autom., Comput. Renew. Syst. (ICACRS)*, Dec. 2022, pp. 813–819.
- [40] J. J. Yebes, D. Montero, and I. Arriola, "Learning to automatically catch potholes in worldwide road scene images," *IEEE Intell. Transp. Syst. Mag.*, vol. 13, no. 3, pp. 192–205, Fall. 2021.
- [41] Y. Cha, W. Choi, and O. Büyükoztürk, "Deep learning-based crack damage detection using convolutional neural networks," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 32, no. 5, pp. 361–378, Mar. 2017.
- [42] M. Faramarzi, "Road damage detection and classification using deep neural networks (YOLOv4) with smartphone images," Jun. 2020. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.3627382>
- [43] J. Dhameeskar, S. Aniruthan, R. Karthika, and L. Parameswaran, "Deep learning based detection of potholes in Indian roads using YOLO," in *Proc. Int. Conf. Inventive Comput. Technol. (ICICT)*, Feb. 2020, pp. 381–385.
- [44] V. Pham, C. Pham, and T. Dang, "Road damage detection and classification with Detectron2 and faster R-CNN," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 5592–5601.
- [45] H. Samma, S. A. Suandi, N. A. Ismail, S. Sulaiman, and L. L. Ping, "Evolving pre-trained CNN using two-layers optimizer for road damage detection from drone images," *IEEE Access*, vol. 9, pp. 158215–158226, 2021.
- [46] R. Vishwakarma and R. Vennekant, "CNN model & tuning for global road damage detection," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 5609–5615.
- [47] D. Wang, Z. Liu, X. Gu, W. Wu, Y. Chen, and L. Wang, "Automatic detection of pothole distress in asphalt pavement using improved convolutional neural networks," *Remote Sens.*, vol. 14, no. 16, p. 3892, Aug. 2022.
- [48] I. D. Pratama, H. Mahmudah, and R. W. Sudibyo, "Design and implementation of real-time pothole detection using convolutional neural network for IoT smart environment," in *Proc. Int. Electron. Symp. (IES)*, Sep. 2021, pp. 675–679.
- [49] I. Moazzam, K. Kamal, S. Mathavan, S. Usman, and M. Rahman, "Metrology and visualization of potholes using the Microsoft Kinect sensor," in *Proc. 16th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2013, pp. 1284–1291.
- [50] H. Brunken and C. Gühmann, "Road surface reconstruction by stereo vision," *PFG – J. Photogramm., Remote Sens. Geoinf. Sci.*, vol. 88, no. 6, pp. 433–448, Dec. 2020.
- [51] Y. Pan, X. Zhang, G. Cervone, and L. Yang, "Detection of asphalt pavement potholes and cracks based on the unmanned aerial vehicle multispectral imagery," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 10, pp. 3701–3712, Oct. 2018.
- [52] S. Faghih-Roohi, S. Hajizadeh, A. Núñez, R. Babuska, and B. De Schutter, "Deep convolutional neural networks for detection of rail surface defects," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2016, pp. 2584–2589.
- [53] L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu, and M. Pietikäinen, "Deep learning for generic object detection: A survey," *Int. J. Comput. Vis.*, vol. 128, no. 2, pp. 261–318, Feb. 2020.

- [54] A. M. Roy and J. Bhaduri, "Real-time growth stage detection model for high degree of occultation using DenseNet-fused YOLOv4," *Comput. Electron. Agricult.*, vol. 193, Feb. 2022, Art. no. 106694.
- [55] B. Jiang, S. Chen, B. Wang, and B. Luo, "MGLNN: Semi-supervised learning via multiple graph cooperative learning neural networks," *Neural Netw.*, vol. 153, pp. 204–214, Sep. 2022.
- [56] A. M. Roy and J. Bhaduri, "DenseSPH-YOLOv5: An automated damage detection model based on DenseNet and swin-transformer prediction head-enabled YOLOv5 with attention mechanism," *Adv. Eng. Informat.*, vol. 56, Apr. 2023, Art. no. 102007.
- [57] A. M. Roy, J. Bhaduri, T. Kumar, and K. Raj, "WilDect-YOLO: An efficient and robust computer vision-based accurate object localization model for automated endangered wildlife detection," *Ecol. Informat.*, vol. 75, Jul. 2023, Art. no. 101919.
- [58] A. Singh, K. Raj, T. Kumar, S. Verma, and A. Roy, "Deep learning-based cost-effective and responsive robot for autism treatment," *Drones*, vol. 7, no. 2, p. 81, Jan. 2023.
- [59] A. M. Roy, R. Bose, and J. Bhaduri, "A fast accurate fine-grain object detection model based on YOLOv4 deep neural network," *Neural Comput. Appl.*, vol. 34, no. 5, pp. 3895–3921, Mar. 2022.
- [60] M. A. K. Raiaan, S. Sakib, N. M. Fahad, A. A. Mamun, M. A. Rahman, S. Shatabda, and M. S. H. Mukta, "A systematic review of hyperparameter optimization techniques in convolutional neural networks," *Decis. Anal. J.*, vol. 11, Jun. 2024, Art. no. 100470.
- [61] S. Shekhar, A. Bansode, and A. Salim, "A comparative study of hyperparameter optimization tools," in *Proc. IEEE Asia-Pacific Conf. Comput. Sci. Data Eng. (CSDE)*, Dec. 2021, pp. 1–6.
- [62] S. C. Smithson, G. Yang, W. J. Gross, and B. H. Meyer, "Neural networks designing neural networks: Multi-objective hyper-parameter optimization," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–8.
- [63] H. Cui and J. Bai, "A new hyperparameters optimization method for convolutional neural networks," *Pattern Recognit. Lett.*, vol. 125, pp. 828–834, Jul. 2019.
- [64] S. Watanabe, "Tree-structured Parzen estimator: Understanding its algorithm components and their roles for better empirical performance," 2023, *arXiv:2304.11127*.
- [65] S. Watanabe, N. Awad, M. Onishi, and F. Hutter, "Multi-objective tree-structured Parzen estimator meets meta-learning," in *Proc. 6th Workshop Meta-Learn. Conf. Neural Inf. Process. Syst.*, 2022.
- [66] Y. Ozaki, Y. Tanigaki, S. Watanabe, M. Nomura, and M. Onishi, "Multiobjective tree-structured Parzen estimator," *J. Artif. Intell. Res.*, vol. 73, pp. 1209–1250, Apr. 2022.
- [67] S. Sieradzki and J. Mańdziuk, "Modified adaptive tree-structured Parzen estimator for hyperparameter optimization," 2025, *arXiv:2502.00871*.
- [68] S. Khessiba, A. G. Blaeich, A. B. Abdallah, A. Manzanera, K. B. Khalifa, and M. H. Bedoui, "A novel hybrid grid search and tree Parzen estimator for deep learning hyperparameters optimization," in *Proc. IEEE/ACS 21st Int. Conf. Comput. Syst. Appl. (AICCSA)*, Oct. 2024, pp. 1–8.
- [69] R. Ishimwe, P. Iradukunda, and J. B. Kwizera, "Real-time road damage detection using deep convolutional neural networks and a smartphone: Project report," Carnegie Mellon Univ., Africa, Tech. Rep., 2021. [Online]. Available: https://www.mininfra.gov.rw/fileadmin/user_upload/Mininfra/Publications/Reports/Project_Report_Real-Time_Road_Damage_Detection_Using_Deep_Convolutional_Neural_Networks_and_a_Smartphone.docx.pdf
- [70] H. Mahmudah, S. Arifin, A. S. Aisjah, and C. A. Prastyanto, "Optimizers impact on RetinaNet model for detecting road damage on edge device," in *IEEE MTT-S Int. Microw. Symp. Dig.*, Feb. 2024, pp. 1–6.
- [71] M. A. Qurishee, "Low-cost deep learning uav and raspberry pi solution to real time pavement condition assessment," Univ. Tennessee at Chattanooga, Chattanooga, TN, USA, Tech. Rep., 2019. [Online]. Available: <https://scholar.utc.edu/theses/601/>
- [72] S.-A. Hassan, T. Rahim, and S.-Y. Shin, "An improved deep convolutional neural network-based autonomous road inspection scheme using unmanned aerial vehicles," *Electronics*, vol. 10, no. 22, p. 2764, Nov. 2021.
- [73] L. Zhang, X. Du, R. Zhang, and J. Zhang, "A lightweight detection algorithm for unmanned surface vehicles based on multi-scale feature fusion," *J. Mar. Sci. Eng.*, vol. 11, no. 7, p. 1392, Jul. 2023.
- [74] J. Jeon, J.-K. Kang, and Y. Kim, "Filter pruning method for inference time acceleration based on YOLOX in edge device," in *Proc. 19th Int. SoC Design Conf. (ISOCC)*, Oct. 2022, pp. 354–355.
- [75] A. Demidovskij and E. Smirnov, "Effective post-training quantization of neural networks for inference on low power neural accelerator," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–7.
- [76] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proc. IEEE*, vol. 111, no. 1, pp. 42–91, Jan. 2023.
- [77] R. D. Rachmanto, Z. Sukma, A. N. L. Nabhaan, A. Setyanto, T. Jiang, and I. K. Kim, "Characterizing deep learning model compression with post-training quantization on accelerated edge devices," in *Proc. IEEE Int. Conf. Edge Comput. Commun. (EDGE)*, Jul. 2024, pp. 110–120.



HANIAH MAHMUDAH (Member, IEEE) received the bachelor's degree from the Engineering Physics Department, and the master's degree from the Electrical Engineering Department, Institut Teknologi Sepuluh Nopember, Indonesia. She is currently a Lecturer with the Electrical Engineering Department, Politeknik Elektronika Negeri Surabaya (Electronics Engineering Polytechnic Institute of Surabaya), Indonesia. Her research interests include signal processing, wire and wireless communication, the Internet of Things, and applied of machine learning. She is a member of ASIOTI and Mastel.



AULIA SITI AISJAH (Member, IEEE) received the Ph.D. degree from the Institut Teknologi Sepuluh Nopember, Indonesia. She is currently a Lecturer with the Engineering Physics Department, Institut Teknologi Sepuluh Nopember. Her research interests include applied of machine learning in industrial, marine engineering, and instrumentation. She is an assessor in the engineering science program at the ASEAN University Network-Quality Assurance (AUN-QA) Network, which was established to carry out the purpose of unifying educational standards, and a member of Indonesian warship consortium.



SYAMSUL ARIFIN (Member, IEEE) is currently a Lecturer with the Engineering Physics Department, Institut Teknologi Sepuluh Nopember, Indonesia. His research interests include instrumentation and automatic controls.



CATUR ARIF PRASTYANTO received the bachelor's, master's, and Ph.D. degrees from the Civil Engineering Department, Institut Teknologi Sepuluh Nopember, Indonesia. He is currently a Lecturer with the Civil Engineering Department, Institut Teknologi Sepuluh Nopember.

...